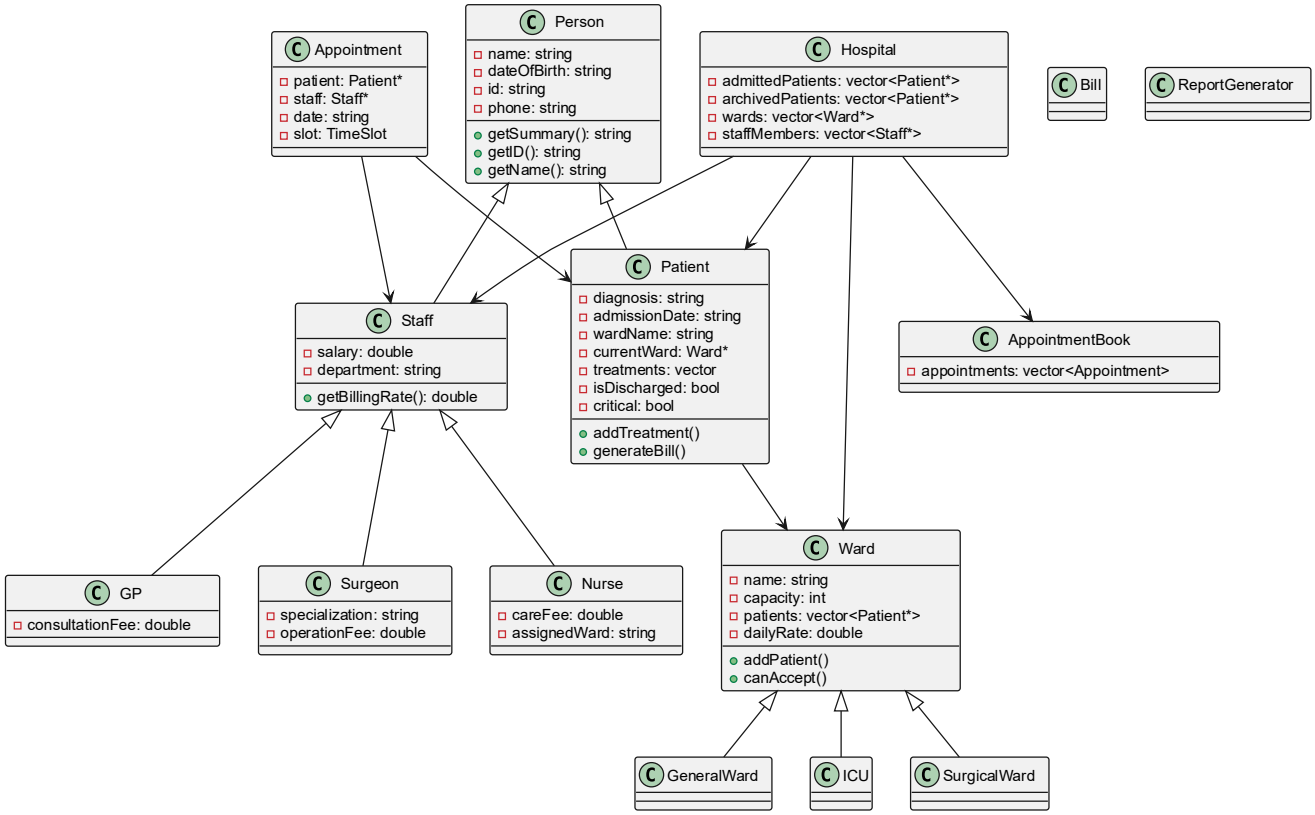
Medicore

Assignment 2
Ali Awj – 32455

1. UML Diagram



3. Class Diagram

Class Diagram Explanation

The system is designed using an object-oriented approach with a strong emphasis on inheritance, polymorphism, and composition.

At the top of the hierarchy is the Person class, which serves as a base class for both Staff and Patient. This design avoids duplication of common attributes such as name, ID, date of birth, and phone number. Both Patient and Staff inherit these shared properties while extending them with their own specialized behaviors.

The Staff class is further specialized into GP, Surgeon, and Nurse, each implementing its own version of getBillingRate(). This allows different staff members to calculate their charges based on their specific roles without relying on conditional logic.

The Patient class stores medical information, including diagnosis, treatments, and admission details. It also maintains an association with a Ward, representing the patient's current location within the hospital.

The Ward class is designed as an abstract base class. It is extended by GeneralWard, ICU, and SurgicalWard, each implementing its own admission criteria. This ensures that different types of wards enforce different rules in a clean and extensible manner.

The Hospital class acts as the central controller of the system. It is responsible for managing:

- Patients (both admitted and archived)
- Staff members
- Wards
- Appointment scheduling through the AppointmentBook

The Appointment class represents the relationship between a Patient and a Staff member, while the AppointmentBook ensures proper scheduling and prevents conflicts such as double booking.

Finally, the ReportGenerator class provides utility functions for filtering, sorting, and calculating statistics such as ward revenue.

4. Design Decisions

1. Use of Base Class (Person)

The Person class is introduced to avoid repetition of common attributes like name, ID, date of birth, and phone number. Both Patient and Staff share these properties, so inheritance is used to maintain a clean and reusable design.

This also enables polymorphism, allowing objects of different types (Patient, GP, Surgeon, Nurse) to be handled through a common interface.

2. Abstract Class for Staff

The Staff class is designed as an abstract class because different staff types have different billing mechanisms.

- GP → consultation fee
- Surgeon → operation fee
- Nurse → care fee

Instead of using multiple if-else conditions, a pure virtual function getBillingRate() is defined and overridden in each derived class. This ensures:

- Clean design
- Easy extensibility
- No type checking required

3. Abstract Class for Ward (Template Method Pattern)

The Ward class is also abstract because each ward has different admission rules.

- ICU → accepts only critical patients
- GeneralWard → accepts all patients
- SurgicalWard → accepts surgery-related cases

The addPatient() function defines the general workflow, while canAccept() is overridden in derived classes. This follows the Template Method Pattern, ensuring:

- Consistent admission logic
- Flexible specialization

4. Polymorphism Instead of dynamic_cast

Instead of using dynamic_cast to check object types, the system uses runtime polymorphism through virtual functions like getBillingRate().

This approach:

- Improves readability
- Reduces coupling
- Follows best OOP practices

5. Value Semantics for Patient Data

The Patient class uses std::vector to store treatments. This ensures:

- Automatic memory management
- No need for manual deletion
- Safer and cleaner implementation

6. Deep Copy and Move Semantics

The Patient class supports both:

- Copy semantics → ensures independent copies
- Move semantics → improves performance during transfers

This is important because treatment data can be large. Move operations allow efficient transfer without copying all elements.

7. Lambda-Based Filtering and Sorting

The system uses lambda functions with std::function for filtering and sorting patients and staff.

Benefits:

- Flexible queries
- No need to modify existing classes
- Supports dynamic behavior from main()

8. Operator Overloading in Billing

The Bill class overloads:

- + → combine bills
- == → compare bills

This makes the system more intuitive and aligns with real-world financial operations.

9. Separation of Concerns

Each class has a clearly defined responsibility:

- Hospital → overall system management
- Patient → medical records
- Ward → admission logic
- ReportGenerator → analytics and reports
- AppointmentBook → scheduling

This improves:

- Maintainability
- Readability
- Scalability

10. Centralized Control via Hospital Class

The Hospital class acts as the main controller, managing all core components:

Patients

Staff

Wards

Appointments

This avoids scattered logic and ensures a well-organized system structure.

5. Memory Ownership Diagram

Memory Ownership Table

Resource	**Owner**	**Allocation**	**Deletion**
Ward Objects	Hospital	main (new)	Hospital destructor
Staff Objects	Hospital	main (new)	Hospital destructor
Patient Objects	Hospital	admit() (new)	Hospital destructor
Patients	main()	new	Deleted in main
Treatment Data	Patient	vector	automatic
Appointment Objects	AppointmentBook	vector	automatic

Ownership Rules

Hospital OWNS:

- Staff objects
- Ward objects
- Admitted patients
- Archived patients

Patient OWNS:

- Treatment data

AppointmentBook OWNS:

- Appointments